

SMART MOSQUITO CONTROL SYSTEM

Project ID: 25-26J-369

Project Proposal Report

Gihan Madhubhashana

BSc. (Hons) Degree in Information Technology Specializing in
Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology, Sri Lanka

August 2025

**SMART MOSQUITO CONTROL SYSTEM –
MOBILE/WEB HEALTH DATA INTEGRATION
PLATFORM**

25 – 26J – 369

Project Proposal Report

Rathnayaka R.M.G.M – IT22568946

Supervisors - Ms. Anjalie Gamage

Co-Supervisor – Mr. Dammika De Silva

BSc (Hons) Degree in Information Technology

Specialized in Information Technology

Department of Information Technology

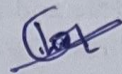
Sri Lanka Institute of Information Technology

Sri Lanka

August 2025

Declaration

We declare that this is our own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Name	Student ID	Signature
Rathnayaka R.M.G.M	IT22568946	

The supervisor/s should certify the proposal report with the following declaration.

The above candidates are carrying out research for the undergraduate Dissertation under my supervision.

Signature of the supervisor:

Date

Signature of the supervisor:

Date

Abstract

Increasing prevalence of mosquito-borne disease in Sri Lanka demands a transition towards proactive instead of reactive public health intervention. This project phase reflects the acute demand for real-time, data-driven mosquito reporting in a vision to innovate and implement a scalable IoT sensor network and cloud data analytics platform. The aim is to implement ESP32-driven sensor nodes as components of smart traps to record environmental data (temperature, humidity) and frequency of mosquito sightings. Data are streamed using MQTT protocol to a cloud infrastructure. Data will be kept in a time-series database (InfluxDB), and a Node.js backend will algorithmically convert the data to supply real-time location-clustered heatmaps of mosquito density. Heatmaps and trends will be rendered over a React-driven dashboards, and actionable intelligence will be generated for health authorities and general public to facilitate targeted intervention. Implementation of this system will bring a reality of a new, real-time analytics platform at the heart of the general goal of reducing vector-borne transmission of disease in Sri Lankan cities.

Table of Contents

Contents

Declaration.....	3
Abstract	4
1.Introduction.....	6
2. Background and Literature Survey.....	7
3. Research Gap	8
4. Research Problem	10
5. Objectives	11
5.1 Main Goal:	11
5.2 Specific Goals:.....	11
6. Methodology	12
6.1 Requirements Analysis ; Technology Selection.	12
6.2 Prototyping & Unit Testing	12
6.3 System Integration & Testing	12
6.4 Field Testing & Rollout	13
6.5 Evaluation & Refinement	13
7. Project Requirements	14
7.1 Functional Requirements:	14
7.2 Non-Functional Requirements:	14
7.3 System Requirements:	14
8. Timeline.....	15
9. References	17

1.Introduction

In Sri Lanka, diseases spread by mosquitoes, like dengue, are a serious and ongoing health crisis, especially in crowded cities. The usual ways of controlling mosquitoes, like fogging, are not very effective. They are used after people are already sick, not to prevent the outbreak in the first place. This is because health officials lack a system that shows them exactly where and when mosquitoes are appearing, in real-time.

This project aims to solve that problem by building a smart, high-tech system. This part of the project focuses on creating the "brain" of that system—the network that collects, stores, and shows the data.

The system will use Internet of Things (IoT) technology. Small, low-cost computers (like the ESP32) will be placed inside mosquito traps. These devices will collect information about the environment, like temperature and humidity, and also send a signal when a mosquito is caught.

This data will be sent wirelessly to a cloud platform. There, it will be stored securely and processed. The most important part of this project is the dashboard: a website that takes all this data and turns it into a simple, colour-coded map (a heatmap). This map will show everyone—from health officials to the public—the exact locations with high mosquito activity in real-time.

While other projects use IoT for simple data logging, the novelty of this work is creating a live, actionable map for public health. This system will move mosquito control from a reactive to a proactive process, helping to save lives and resources.

2. Background and Literature Survey

The use of Internet of Things (IoT) technology to monitor the environment is now common. Systems exist to track air quality, weather, and water levels using networks of sensors [1]. Big cloud companies like Amazon and Microsoft offer powerful platforms (AWS IoT Core, Azure IoT Hub) that make it easy to connect thousands of devices and manage the data they send [2]. These platforms handle the complex tasks of security, communication, and data ingestion, allowing developers to focus on building their applications.

A key technology for IoT is the MQTT protocol. It is designed to be very efficient, making it perfect for small, battery-powered devices like our ESP32 microcontroller [3]. Unlike a standard web request, MQTT uses a "publish-subscribe" model. This means our sensor device can "publish" a temperature reading to a central broker, and our dashboard can "subscribe" to that broker to receive the new data instantly. This is much faster and uses less power than constantly checking a website for updates.

Choosing the right database is critical. Sensor data is a stream of measurements, each with a timestamp. While a general-purpose database like MongoDB is flexible and can store this JSON-like data easily, a time-series database (TSDB) like InfluxDB is built specifically for this job [4]. It excels at storing and retrieving huge amounts of time-stamped data very quickly. It performs much better for queries like "show me the average temperature every 5 minutes for the last week," which is exactly what our dashboard needs to draw its graphs and heatmaps.

Finally, to show this data to users, we need a modern web dashboard. JavaScript libraries like React are the industry standard for building dynamic and interactive web applications [5]. When combined with a mapping library like Leaflet [6], we can create real-time maps that visually represent data. For our project, this means plotting mosquito trap locations and drawing colour-coded heatmaps to show high-risk areas, allowing users to understand the situation instantly.

While all these technologies are mature, a clear research gap exists in their combined use for public health. Many studies use IoT for generic environmental sensing [7]. However, there are few examples of a fully integrated system that uses MQTT, a cloud database, and a React dashboard specifically for creating a real-time analytics engine to track mosquitoes [8]. This project will fill that gap by building a complete solution tailored to this urgent health problem.

3. Research Gap

Current research into IoT applications for public health often focuses on broad environmental monitoring, such as tracking general air quality or weather patterns [10]. However, a significant gap exists when these technologies are applied to the specific, critical problem of mosquito-borne diseases [12].

Many existing systems operate as data silos. They may collect data effectively, but they lack a seamless integration to transform that raw data into actionable intelligence [13]. For instance, a trap might count mosquitoes, but that data is not automatically sent to a central system. Alternatively, data might be stored but not processed in real-time to generate useful visuals for decision-makers [14].

Furthermore, there is a notable lack of research on building integrated, real-time analytics platforms that perform several key functions together [15]. These functions include: 1) collecting entomological (mosquito) and environmental (temperature, humidity) data from sensors in the field; 2) processing these data streams in real-time; and 3) transforming it into visual intelligence (like dynamic heatmaps) on a dashboard [16].

This gap means that health officials often cannot see outbreak patterns as they are forming. Most current solutions do not provide a live, intuitive public health dashboard that shows where mosquitoes are most active right now [17]. Without this, efforts to control mosquitoes remain reactive instead of proactive [18].

Therefore, the research gap this project addresses is not just using technology, but the integration of specific technologies—IOT protocols, cloud databases, and visualization libraries—into a single, cohesive system [19]. This system is designed to close the loop from data collection to visual insight, specifically for predicting and mitigating mosquito-borne disease outbreaks [20].

Research Feature / Capability	Standalone Smart Traps	IoT Data Loggers	Cloud Dashboards with Historical Data	Our Proposed Integrated System
Real-time Data Transmission	No (Data stored locally)	Yes (But often uses HTTP)	Yes	Yes (Using efficient MQTT)
Mosquito Species Identification	Yes (On-device AI)	No (Only environmental sensors)	No	Yes (Via integration with AI module)
Live Environmental Sensing	Limited (Focus on trap function)	Yes (Primary function)	Yes (Data from loggers)	Yes (Integrated sensor suite)
Real-time Cloud Database	No	No (Simple storage)	Yes (But often SQL, not optimized)	Yes (Time-series optimized - InfluxDB/MongoDB)
Dynamic, Live Heatmaps	No	No	No (Static or historical maps only)	Yes (Core feature of dashboard)
Public Health Integration	No	No	Limited (Officials-only access)	Yes (Web dashboard for officials & public)
End-to-end Automation	No (Manual data retrieval)	No (Manual analysis needed)	Partial (Manual reporting)	Yes (Fully automated from trap to map)

4. Research Problem

The main problem this research addresses is that current mosquito monitoring systems don't work together effectively. While there are many tools available, they don't connect properly to provide real-time information that health workers can use immediately.

There are three specific problems we need to solve:

- **Separated Data:** Different devices collect different information (like trap counts, temperature, and location), but this data stays separate. We need to create a system that can combine all this information into one clear picture.
- **Slow Information:** Even when data is collected, it takes too long to process. By the time reports are ready, the information is already outdated. We need to make data available immediately.
- **Hard-to-Understand Results:** The data isn't presented in a clear way. Health officials need simple maps and charts that show where mosquitoes are most active right now, not complicated spreadsheets.

In simple terms, we need to create a complete system that collects data from various sensors, processes it quickly, and shows the results on easy-to-understand maps - all in real time. This will help health workers take action before diseases spread.

5. Objectives

5.1 Main Goal:

To design, develop and deploy a complete, robust, end-to-end IoT sensor network and cloud analytics dashboard. This can facilitate the real-time monitoring, storage, and visualization of mosquito density and other relevant environmental data, thus providing the main intelligence platform of the smart mosquito control system.

5.2 Specific Goals:

To prototype and test a dependable sensor node based on the ESP32 microcontroller. This includes writing the firmware necessary to read data from environmental sensors (e.g., DHT22 for temperature and humidity). The node must connect to Wi-Fi reliably and publish sensor readings as structured JSON data to a central MQTT broker at intervals allowing for reliable and low power data transfer.

To architect, build and secure a complete cloud infrastructure to consume and manage incoming data. This includes configuring a dedicated MQTT broker to consume and manage each message produced by the IoT sensor network.

Additionally, it will implement a time-series db (e.g., InfluxDB) or structured db (e.g., MongoDB) to appropriately store and organize an influx of time-stamped data from the sensors, a secure backend API written in Node.js and Express.js to serve as the interface for incoming requests, process the data, and provide all data to the front-end dashboard and other services.

6. Methodology

The IoT sensor network as well as analytics dashboards construction will use a systematic agile methodology in five consecutive stages. With such an approach, there is systematic progression while there remains a pace for iteration for improvement based on testing in addition to feedback.

6.1 Requirements Analysis ; Technology Selection.

This first phase will involve laying a complete technical foundation. Certain tasks will involve: performing an in-depth analysis of information needs for each system component; comparing competing technologies (e.g., weighing HiveMQ Cloud versus AWS IoT Core for an MQTT broker comparison, or comparing InfluxDB versus MongoDB for a database comparison); solidifying a complete system architecture diagram; and determining detailed data schemes for MQTT messages and database collections. As a result, there will be a complete technology stack and detailed system spec report.

6.2 Prototyping & Unit Testing

It consists of simultaneous development and testing of separate system components. Certain tasks are: developing and testing sensor measurement achievement and wireless transmission ESP32 firmware; setting up a Python-oriented MQTT bridge for receiving messages and storing in the database; designing database schema and setting up basic query functions; developing Node.js API endpoints for extracting information; and constructing simple React components for information display. Each component is strictly tested with simulated information for correct behavior prior to integration.

6.3 System Integration & Testing

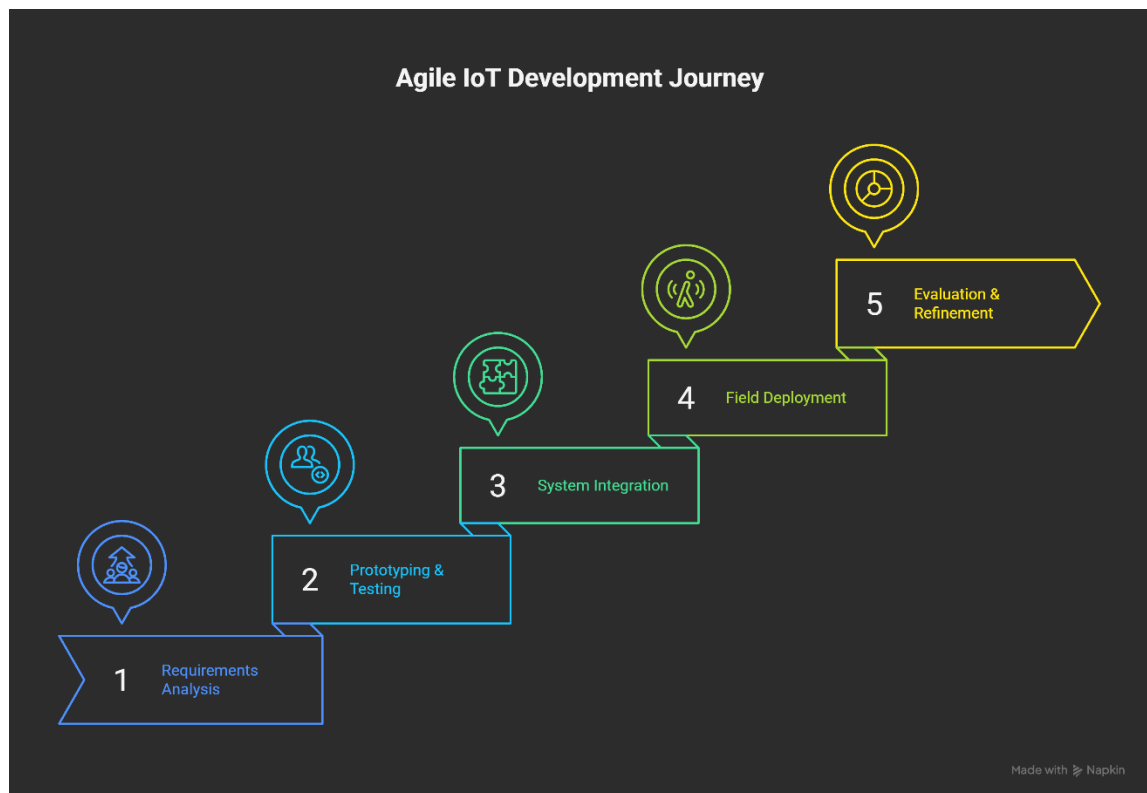
This integration phase is about putting all parts together in a coherent system. Tasks involve: setting up end-to-end connectivity between ESP32 prototypes, MQTT broker, database, and dashboard; setting up end-to-end data transmission flow from sensor to visualization; performing integration testing for confirming cohesiveness in the system; subjecting the system to load testing for confirming its ability to support multiple data streams at a time; and confirming correctness in the data throughout its entire pipeline. The result shall be an integrated ready-to-deploy system.

6.4 Field Testing & Rollout

This phase transfers the system from prototype construction to real-world operation. Some tasks are: deploying the cloud infrastructure into a production environment; setting up safe access controls for the dashboard; interfacing with hardware team functional trap prototypes; performing controlled field testing with physical units; observing system behavior in real-world conditions; and obtaining first performance metrics. Deliverables will be a completely deployed system handling real sensor input. Phase

6.5 Evaluation & Refinement

The final phase is optimization and concluding delivery preparations. Work involved: performing complete system assessment against all objectives; gathering and incorporating stakeholders' responses; implementing performance optimizations based on test outcomes; user interface tuning for increased usability; finalization of all documentation; and preparations for system handover. Deliverables shall be a polished, complete system completely satisfying all specified requirements.



7. Project Requirements

7.1 Functional Requirements:

- The system shall insert sensor data (temperature, humidity, device ID) from traps via MQTT.
- The system shall store ingested sensor data in a time-series database.
- The system shall display real-time mosquito density data on a map as a heatmap overlay.
- The system shall display historical trends of environmental data on interactive graphs.
- The system shall provide a RESTful API for the mobile app to fetch heatmap data.

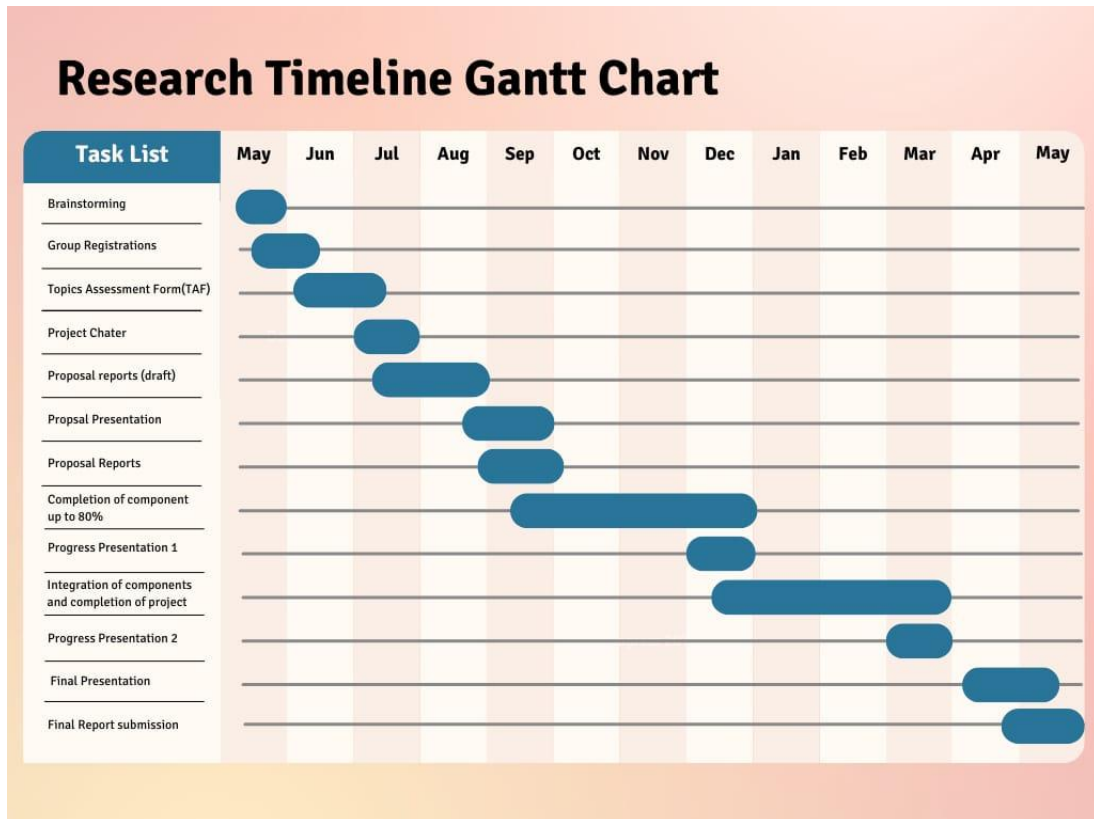
7.2 Non-Functional Requirements:

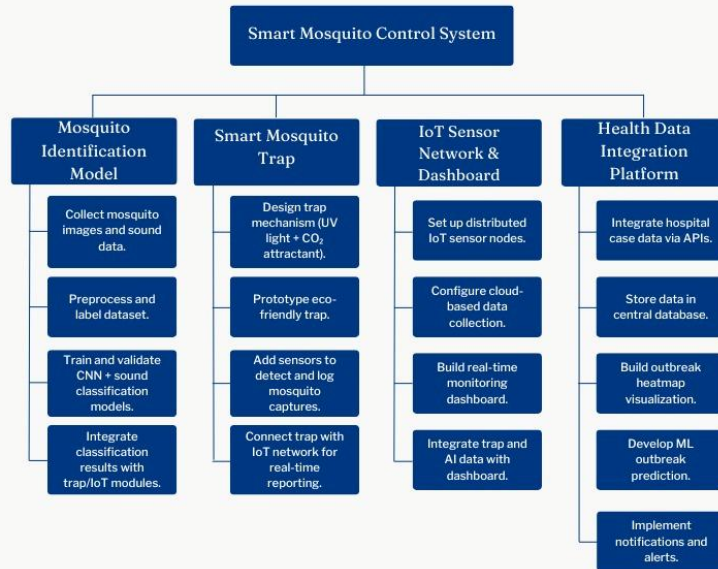
- **Performance:** The dashboard shall update within 5 seconds of new data being received.
- **Scalability:** The system architecture shall support scaling to 100+ sensor nodes.
- **Availability:** The cloud dashboard shall have 99% uptime.
- **Usability:** The dashboard interface shall be intuitive and require minimal training.

7.3 System Requirements:

- **Frontend:** React.js, Axios
- **Backend:** Node.js, Express.js
- **Database:** InfluxDB, MongoDB
- **IoT Protocol:** MQTT
- **Hardware:** ESP32 Microcontroller
- **Cloud:** HiveMQ Cloud, A cloud provider for hosting (AWS EC2)

8. Timeline





9. References

- [1] J. Gubbi, R. Buyya, S. Marusic, and M. Palaniswami, "Internet of Things (IoT): A vision, architectural elements, and future directions," *Future Gener. Comput. Syst.*, vol. 29, no. 7, pp. 1645-1660, 2013.
- [2] Amazon Web Services, "AWS IoT Core Developer Guide," 2023. [Online]. Available: <https://docs.aws.amazon.com/iot/>
- [3] A. Banks and R. Gupta, "MQTT Version 5.0," OASIS Standard, 2019.
- [4] InfluxData, "InfluxDB documentation," 2023. [Online]. Available: <https://docs.influxdata.com/influxdb/>
- [5] React Community, "React Documentation," 2023. [Online]. Available: <https://reactjs.org/docs/>
- [6] V. Agafonkin, "Leaflet: An open-source JavaScript library for interactive maps," 2023. [Online]. Available: <https://leafletjs.com/>
- [7] M. N. K. Boulos and N. M. Al-Shorbaji, "On the Internet of Things, smart cities and the WHO Healthy Cities," *Int. J. Health Geogr.*, vol. 13, no. 1, p. 10, 2014.
- [8] L. M. Hernández et al., "A Review of IoT Systems for Mosquito-Borne Disease Surveillance and Control," *IEEE Sens. J.*, vol. 22, no. 18, pp. 17361-17371, 2022.